Practical - 3

Practical Title: Project Initiation, Requirement Analysis, Sitemap, Wireframe, and GitHub Setup

Problem Definition: Initiate the semester-long StudentHub portal by identifying the problem scope, user roles, key modules, navigation flow, and minimum 10 pages. Create a sitemap, low-fidelity wireframe, project folder structure, README file, and GitHub repository.

A. Theory

1. How does the layout adapt for mobile, tablet, and desktop screens?

Answer: The layout adapts using CSS media queries. I have set breakpoints so that on desktop screens the "Main Pages" links show as a multi-column grid of cards using flexbox with wrapping, while on smaller tablet and mobile widths the flex-wrap property lets the cards drop down into fewer columns automatically instead of overflowing. The navigation bar links also wrap onto the next line on narrow screens instead of getting cut off, since I used flex-wrap and padding instead of fixed widths, so the layout reflows rather than breaking on smaller devices.

Reference: https://www.browserstack.com/guide/media-query-for-desktop-tablet-mobile and https://penpot.app/blog/how-to-use-css-and-media-query-breakpoints-to-create-responsive-layouts/

2. Which layout technique is used and why?

Answer: I mainly used Flexbox for the layout, for example the .home-links list on the homepage uses display: flex with flex-wrap so the page cards line up in a row and wrap to the next line on smaller screens, and the nav bar links are laid out the same way. I chose Flexbox over Grid because my layout is mostly one-dimensional (a single row of cards or links that needs to wrap), and Flexbox is simpler to set up for that kind of content-driven layout compared to defining explicit grid tracks, which I read is more suited to complex two-dimensional layouts.

Reference: https://blog.logrocket.com/css-flexbox-vs-css-grid/ and https://developer.mozilla.org/en-US/docs/Web/CSS/Guides/Grid_layout/Relationship_with_other_layout_methods

3. Are typography, spacing, colors, and contrast readable?

Answer: Yes, I tried to keep these readable. The body uses a simple sans-serif font (Arial) with a line-height of 1.6 so text does not feel cramped, and headings use a larger size with the same blue accent color (#4a90d9) so they stand out from normal paragraph text. For spacing I used consistent padding and margins around headings, paragraphs and list items

instead of letting them touch the edges of the page. For contrast, the white login/registration form boxes sit on a light grey background, and the header/nav use white text on a darker blue/black background, which keeps the text readable rather than low-contrast text on a similar colored background.
Reference: https://pxlpeak.com/blog/web-design/website-accessibility-wcag-guide and https://www.w3.org/TR/WCAG22/

4. Is the framework used consistently without unnecessary inline CSS?
Answer: Yes, all the styling for every page comes from one shared external stylesheet (script.css) that is linked in the head of each HTML page, so the same classes like .home-links and the same tag-based rules for nav, form, input and button are reused everywhere instead of repeating styles per page. I avoided inline style="..." attributes on individual elements and instead used tag selectors, one class (.home-links) and pseudo-classes like :hover and :focus in the CSS file, which keeps the HTML clean and makes it easy to change the look of the whole site by editing the CSS in one place.
Reference: https://www.w3schools.com/css/css_howto.asp and https://developer.mozilla.org/en-US/docs/Web/CSS/CSS_selectors

B. Implementation/ Output/Result

```
* {
  margin: 0;
  padding: 0;
  box-sizing: border-box;
}

body {
  font-family: Arial, sans-serif;
  background-color: #f2f2f2;
  color: #333;
  line-height: 1.6;
}

h1 {
  background-color: #4a90d9;
  color: white;
  text-align: center;
  padding: 20px;
  font-size: 28px;
}

h2 {
  color: #4a90d9;
  margin-top: 25px;
  margin-bottom: 15px;
```

```
  padding-left: 20px;
  font-size: 22px;
  border-bottom: 2px solid #4a90d9;
  padding-bottom: 5px;
  margin-left: 20px;
  margin-right: 20px;
}

nav {
  background-color: #333;
  padding: 10px;
  text-align: center;
}

nav a {
  color: white;
  text-decoration: none;
  padding: 8px 14px;
  margin: 2px;
  display: inline-block;
  font-size: 14px;
}

nav a:hover {
  background-color: #4a90d9;
  border-radius: 4px;
}

p {
  padding: 5px 25px;
  font-size: 16px;
  margin-bottom: 8px;
}

a {
  color: #4a90d9;
  text-decoration: none;
}

a:hover {
  text-decoration: underline;
}

ul {
  list-style-type: square;
  margin-left: 45px;
  margin-right: 20px;
  margin-bottom: 20px;
}

ul li {
  padding: 6px 0;
  font-size: 16px;
}
```

```
.home-links {
  list-style-type: none;
  display: flex;
  flex-wrap: wrap;
  gap: 15px;
  padding: 10px 25px;
  margin-left: 0;
}

.home-links li {
  background-color: white;
  border: 2px solid #4a90d9;
  border-radius: 8px;
  width: 180px;
  text-align: center;
  transition: background-color 0.3s, transform 0.2s;
}

.home-links li:hover {
  background-color: #4a90d9;
  transform: scale(1.05);
}

.home-links li a {
  color: #4a90d9;
  font-weight: bold;
  text-decoration: none;
  display: block;
  padding: 20px 10px;
  font-size: 16px;
}

.home-links li:hover a {
  color: white;
}

form {
  background-color: white;
  max-width: 400px;
  margin: 20px auto;
  padding: 25px;
  border: 1px solid #ccc;
  border-radius: 6px;
}

form p {
  padding: 5px 0;
}

label {
  display: inline-block;
  width: 100px;
  font-weight: bold;
```

```
  font-size: 14px;
  color: #555;
}

input[type="text"],
input[type="password"],
input[type="email"] {
  width: 220px;
  padding: 6px 10px;
  border: 1px solid #ccc;
  border-radius: 4px;
  font-size: 14px;
}

input[type="text"]:focus,
input[type="password"]:focus,
input[type="email"]:focus {
  border-color: #4a90d9;
  outline: none;
}

button {
  background-color: #4a90d9;
  color: white;
  padding: 8px 25px;
  border: none;
  border-radius: 4px;
  font-size: 15px;
  cursor: pointer;
}

button:hover {
  background-color: #3a7bc8;
}

strong {
  color: #555;
}

footer {
  background-color: #333;
  color: white;
  text-align: center;
  padding: 10px;
  margin-top: 30px;
  font-size: 13px;
}
```

C. Evidence of Work (Screenshots)

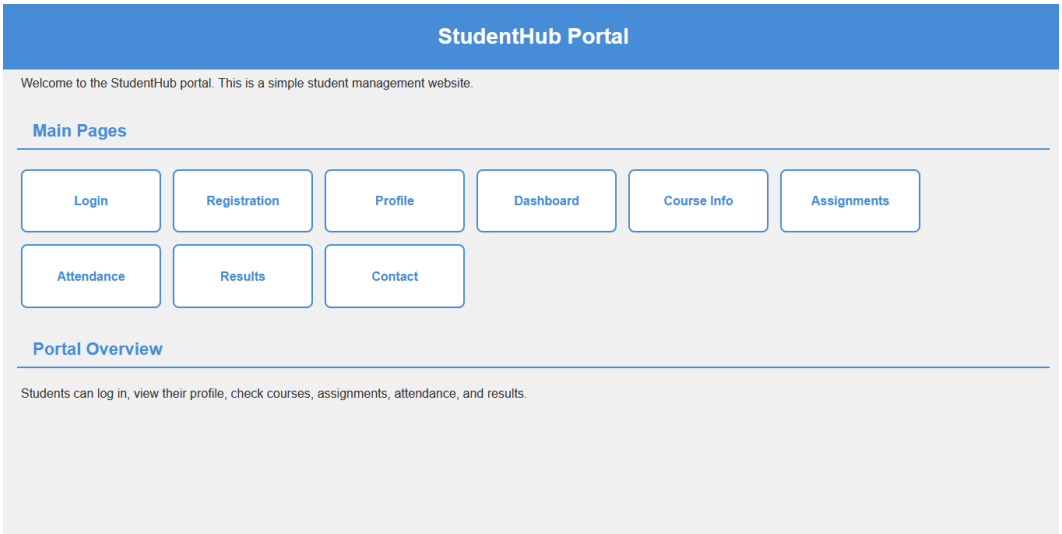


Figure 1.1: StudentHub Portal Home Page

Description:

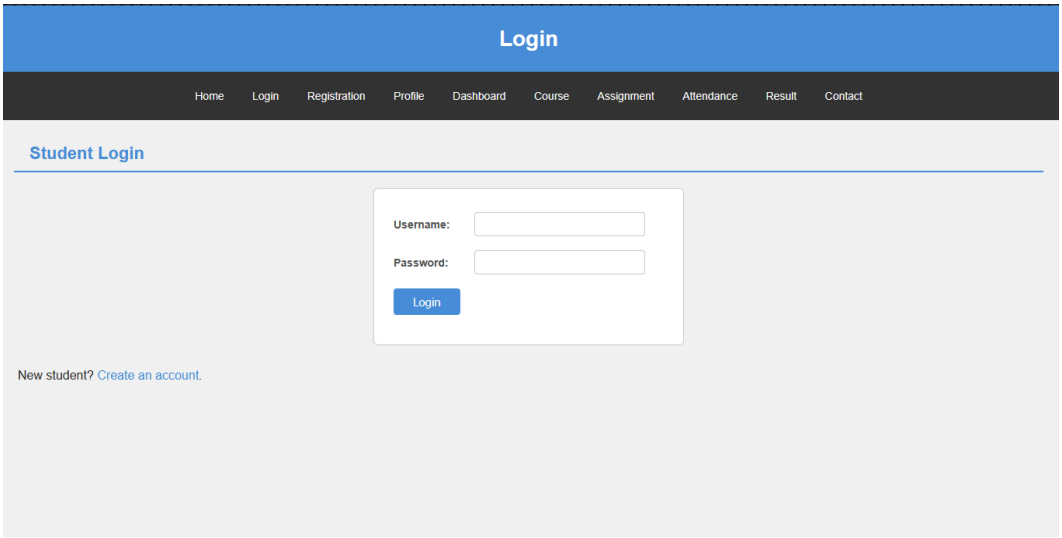


Figure 1.2: Student Login Page

Description:

D. GitHub Repository Link:

https://github.com/VRAJ-710/studenthub-portal

E. Challenges Faced:

I had a difficulty in implementing all css together in a single css file for all pages also with border selection was a bit tricky to figure out by trial and check method. and also i have to continuously look at css selectors and commands too

F. Learning Outcomes:

From this practical I learned how to structure a single external stylesheet that styles multiple HTML pages consistently, instead of writing separate styles per page. I got hands-on practice with Flexbox for building card-style layouts and navigation bars that wrap responsively, and I understood how simple properties like padding, border-radius, box-shadow-like borders, and hover/focus states can make a page feel more polished. I also became more comfortable using browser dev tools to trial-and-check CSS selectors like class selectors, attribute selectors (input[type]) and pseudo-classes, which helped me figure out styling issues faster instead of guessing.

Faculty Remarks: